NORTEX

Language Reference

Version 0.1.0

Table of contents

Chapter 1 – Introduction 5
- What is NORTEX? 5
- Goals 5
- Verified properties 5
- Distinction 5
- Realistic use cases today 5
- Technical requirements 5

Chapter 2 – Installation and Setup 6

Chapter 3 – Language Basics 7
- Program structure 7
- Statements, expressions, and blocks 7
- Comments 7
- Identifiers 8
- Keywords 8
- Typical errors 8
- Naming conventions (recommendation) 8
- Planned (design target) 8

Chapter 4 – Data Types and Variables 9
- Primitive types 9
- Literals 9
- Variables 10
- Type inference 10
- Type conversions 10
- Scopes 10
- Limitations 11
- Planned (design target, not executable) 11

Chapter 5 – Operators 12
- Overview 12
- Precedence (high -> low), associativity 12
- Behavior 12
- Type compatibility 13
- Common errors 13
- Operator overloading 13

Chapter 6 – Control Flow 14
- if / else 14
- while 14
- for 14
- break and continue 14
- return 14
- Ternary 15
- Other constructs (frontend only) 15

Chapter 7 – Functions and Methods 16
- Function definition 16
- Calls and recursion 16

Return values 16
Local variables 16
Methods and visibility (frontend only) 16
Limitations 17
Design rules and frontend status 17
Chapter 8 – Structs and Data Structures 18
Struct definition 18
Initialization and access 18
Classes with methods 18
Other declarations 18
Arrays and collections 18
Nesting 19
Classes, inheritance, generics, collections (design target) 19
Chapter 9 – Memory Management 20
Current status 20
Language goal: Ownership 20
Safety mechanisms (implemented) 20
Lifetime 20
Runtime 20
Planned memory model (design) 20
Chapter 10 – Input and Output 21
Console output 21
Not supported (NX2001/NX2002 in each case) 21
String concatenation 21
Return values as an output channel 22
Standard library 22
Strings (design target) 22
Chapter 11 – Errors and Debugging 23
Format 23
Error classes with codes actually observed 23
Runtime errors 23
Known compiler bug 23
Strategies 24
Error handling (design and frontend status) 24
Asynchrony (planned) 24
Chapter 12 – Compilation and Technical Architecture 25
Pipeline 25
Commands 25
Backend properties 25
Note about older documents 26
ABI (design target) 26
.ntx package format (planned) 26
Chapter 13 – Practical Examples 27
1. Hello World – 01_hello.nx 27

2. Variables and calculations – 02_variables.nx 27
3. Conditions – 03_conditions.nx 27
4. Loops – 04_loops.nx 28
5. Functions and recursion – 05_functions.nx 28
6. Complete application: prime numbers – 06_primes.nx 28
7. Structs – 07_struct_frontend.nx (check only) 29
8. Methods – 08_class_frontend.nx (check only) 29
9. Heap allocation 29

Chapter 14 – Language Reference 30
Keywords 30
Tokens 30
Operators and symbols 30
Grammar constructs (native) 30
Supported statements 31
Built-in functions 31
Diagnostic codes (observed) 31
Relevant limitations 31

Chapter 15 – FAQ and Best Practices 32
Common beginner mistakes 32
FAQ 32
Recommended practices 32
Performance 32
Larger projects 32
Open decisions (project owner) 33

Chapter 1 – Introduction

<- Contents · Next: Chapter 2

What is NORTEX?

NORTEX is a statically and strongly typed, compiled programming language with C#-like declaration syntax. The nortexc compiler is written in Rust and uses Cranelift to produce native Windows x64 programs (.exe).

```
public int main() {
    return 42;
}
```

Goals

According to the project documentation: memory safety without a garbage collector (Ownership/Borrowing), explicit mutability, stable diagnostic codes (NXnnnn), and a compiled execution path. Mutability rules and diagnostics are implemented; ownership checking is partially implemented (see Chapter 9).

Verified properties

- Identifiers and keywords are case-sensitive; identifiers are ASCII-only.
- Bindings are immutable by default; mut makes them mutable.
- No null in safe code.
- Integer overflow checking is enabled by default.
- German compiler messages with stable codes.

Distinction

NORTEX looks like C#, but is an independent language: top-level functions without a class, mut bindings, ownership concepts, and a considerably smaller feature set. Do not assume syntax from other languages (for example, no casts, no do-while, and no let).

Realistic use cases today

Small console programs with integer logic, compiler/language experiments, and learning projects. General application development will require strings, structs, a heap, and a standard library (see README).

Technical requirements

Windows 10/11 x64, the Rust toolchain (Edition 2024, Cargo >= 1.88), and the MSVC Build Tools (link.exe linker). See Chapter 2.

Chapter 2 – Installation and Setup

<- Chapter 1 · Contents · Next: Chapter 3

Requirements (Windows x64, Rust >= 1.88, MSVC Build Tools), build commands, a first program, and the nortex.toml manifest are covered in the central installation guide.

Quick reference:

```
cargo build --release -p nortex-compiler
nortexc build hello.nx -o hello.exe
```

Note: using util; resolves user-defined .nx files (flat namespace); see the implemented specification.

Chapter 3 – Language Basics

<- Chapter 2 · Contents · Next: Chapter 4

Program structure

A source file (.nx) consists of optional using/namespace lines and declarations. Functions may appear directly at the top level. The native entry point must be exactly one parameterless function public int main(); its return value becomes the exit code.

```
using System;

public int main() {
    Console.writeLine("Hello");
    return 0;
}
```

void main(), main(int a), and bool main() are rejected during the build with “Windows-Programme benötigen derzeit genau eine parameterlose int-main-Funktion”; without main, the linker fails (LNK2019).

using System; is not required for Console in the tested path, but is conventional. using with an unknown namespace (using Foo.Bar;) produces NX1003.

Statements, expressions, and blocks

- A statement ends with ; (declaration, assignment, call, return, break, continue).
- An expression produces a value (a * b, f(x), c ? 1 : 2).
- A block { ... } groups statements and creates its own scope.

```
public int main() {
    int a = 1;            // declaration
    mut int b = a + 2;    // expression used as initializer
    if (b > 2) {          // block
        b += 1;
    }
    return b;
}
```

Missing semicolon: error[NX0100]: Nach einer Variablendeklaration wird ';' erwartet. There is no ; after blocks (}).

Comments

```
// Line comment
/* Block comment */
```

Comments may contain Unicode.

Identifiers

[A-Za-z_][A-Za-z0-9_]*, ASCII only. Case matters (name, Name, and NAME are different). Style warnings NX0004: constants should use UPPER_SNAKE_CASE; variables/members should use camelCase (warning, not an error).

Keywords

All reserved words are listed in Chapter 14. get/set are ordinary identifiers.

Typical errors

Error	Message
Unknown character (@)	NX0100 Unbekanntes Zeichen '@'
Reserved let	NX0100 'let' ist reserviert, aber in NORTEX nicht unterstützt
Unknown identifier	NX1004
Duplicate declaration in the same scope	NX1001

Naming conventions (recommendation)

Identifiers consist of ASCII letters, digits, and _; Unicode is allowed only in comments and string/character literals. Use PascalCase for types (without an I prefix for interfaces), camelCase for methods and variables, UPPER_SNAKE_CASE for constants, and snake_case for module/file names. Two-letter acronyms remain uppercase (IOStream, getID); longer ones are treated like words (XmlParser). Private fields do not have leading underscores. Violations are warnings (NX0004), not errors.

Planned (design target)

- using imports only NORTEX namespaces; it has no connection to .NET.
- Visibility modifiers public, private, protected, and internal are parsed; enforcement across module boundaries is missing (see Chapter 14).
- The .ntx package format is part of the design; see Chapter 12.

Chapter 4 – Data Types and Variables

<- Chapter 3 · Contents · Next: Chapter 5

Primitive types

Type	Width	Native (build)
int	32-bit, signed	yes
bool	true/false	yes
uint, short, ulong	32/16/64-bit	yes for locals and parameters/returns (uint function tested)
long	64-bit	yes for parameters/returns; locals with division/comparison can crash codegen (known Cranelift panic)
sbyte, byte, ushort, i128, u128, nint, nuint, isize, usize	see this page (section “Planned”)	frontend; not individually tested natively
float, double, decimal	IEEE-754 32/64 or 128-bit	no (NX2001)
char	Unicode scalar	no (NX2002)
string	UTF-8 slice (data pointer + byte length)	yes, as a literal, local variable, internal parameter/return value, and Console argument
void	no value	return type only

Literals

String literals and their 16-byte slice descriptors reside read-only in the COFF data section. Length is measured in UTF-8 bytes.

- Concatenation: a + b calls the runtime's nortex_string_concat. It reserves a block containing a descriptor, bytes, and a trailing NUL via HeapAlloc, and returns the new slice. If allocation fails, the program terminates with a trap.
- Globals: Top-level const string and mut string are allowed. The initializer must be a string literal or a + concatenation of literals (folded at compile time). The global is a pointer to a descriptor in .rdata; mut string replaces only the slice.
- extern "C": String arguments are passed as char* (data pointer). All strings are NUL-terminated; an embedded \0 terminates the string early from C's perspective. External functions may not return string.
- Comparison: == and != call nortex_string_eq (length and byte comparison) and return bool. Ordering operators (, …) on strings are not implemented.
- Drop and Move: Heap strings (results of +) belong to exactly one variable. At scope exit, on return, and before overwriting, codegen calls nortex_string_free (HeapFree), unless the value was moved. Assignment, initialization, return, and passing to internal functions move the source string; any further use reports NX1202. Concatenation, comparison, Console, and extern "C" calls only borrow. Static literals and const string are never freed.
- Limitations: Moving a mut string global (string b = name;, return, argument, ternary branch) copies the value (name + ""); the global retains its buffer. Reassigning it frees the old heap buffer.

```
int a = 42;
int h = 0xFF;          // hexadecimal
int b = 0b1010;        // binary
double d = 1.5e3;      // exponent (frontend)
```

```
float f = 2.5f;        // f suffix required; 2.5 is double
char c = 'x';
string s = "text";
bool t = true;
```

Literals are checked against the target type: int a = 3000000000; -> NX1102 … is outside the value range of 'int'. Integer suffixes such as 5L lex successfully, but fail in native lowering (NX2002). String escapes: \n \r \t \0 \" \\; unknown escapes (\q) -> NX0100.

Variables

```
int a = 1;          // immutable
mut int b = 2;      // mutable
const int LIMIT = 3;
```

- Immutable by default. a = 2; -> NX1200 Der unveränderliche Wert 'a' darf nicht neu zugewiesen werden.
- Global values may also appear at the top level. They require a compile-time-evaluable initializer; mut allows later changes. Example: public const int MAX_PLAYERS = 100; and mut int playerCount = 0;.
- Global integer and bool values are emitted natively in read-only or writable COFF data sections. Other globally declarable types are checked by the frontend, but are not yet available in the native MIR subset (NX2002).
- Parameters and loop variables are also immutable; for requires mut int i.
- const requires UPPER_SNAKE_CASE (otherwise warning NX0004).
- Every variable needs both a type and an initializer. let is reserved and cannot be used. readonly int a = 1; is invalid as a local declaration.
- null is forbidden (NX1102).

Type inference

var c = a * b; passes check, but is rejected during build: “Variablentyp-Inferenz wird in dieser Lowering-Stufe nicht unterstützt” (NX2001). Always use explicit types in native code.

Type conversions

There are no implicit conversions between integer types: uint + int -> NX1102 Die Operanden von '+' müssen denselben numerischen Typ besitzen; returning a byte from an int function -> NX1102. No working cast syntax is available ((int)x and x as int are not parsed).

Scopes

A block {} creates a scope. Variables are visible from their declaration to the end of the block; use outside that scope -> NX1004. Duplicate names in the same scope -> NX1001. In the tested case, a nested scope cannot reassign an outer name if it is immutable (NX1200).

```
public int main() {
    mut int a = 1;
    if (a == 1) {
        int b = 5;
        a = b;          // ok: a is mutable
```

```
    }
    return a;           // 5; b is not visible here
}
```

Limitations

Only scalar integer/boolean values are native. Strings, floating-point values, arrays, structs, and classes are not yet executable.

Planned (design target, not executable)

Primitive type names are case-sensitive and have no aliases (Int32, String, Bool, Unit, Never are invalid). Implicit lossy numeric conversions are excluded. char is a 32-bit Unicode scalar value; string is UTF-8, and length() counts bytes. void is not a storable value type; the divergent type is !. nint/nuint/isize/usize are pointer-sized (64-bit on Windows x64/ARM64).

Type inference: var x = …; infers the type from the initializer (current design: immutable, use mut var for mutable). Global inference across parameters, returns, and generics is not implemented; ambiguous inference should be an error. const requires a compile-time value.

Decimal128 (design): decimal is intended to be a custom 128-bit fixed-point format: 1 sign bit, 7 scale bits (0–38), and a 120-bit coefficient (0 to 10³?-1); value (-1)^sign x coefficient / 10^scale, with nearest, ties to even rounding; overflow/division by zero should raise ArithmeticException. No silent conversion to double. Only the format is specified; no arithmetic is implemented.

Composite types (design): T[] (dynamic), [T; N] (fixed), tuples (T1, T2, …) with at least two elements, struct, class, record (immutable, structural equality), enum (sum type), interface, trait. Option (Some/None) and Result (Ok/Err); there is no null in the safe model.

Other type concepts (design, names only):

Type	Intended semantics
object	explicit type erasure/boxing, not a universal base type
dynamic	dynamic resolution with runtime checking (DynamicDispatchException)
&T, &mut T	shared or exclusive borrow
*const T, *mut T	raw pointer, dereference only in unsafe
Box	unique heap owner
Rc, Arc	shared ownership (non-thread-safe / atomic)
Cell, RefCell	interior mutability (RefCell with runtime checking)
Mutex, RwLock	synchronization; Send/Sync mark thread safety
Span	non-owning view of contiguous memory
Memory	owning runtime handle
Func, Action	callable values; captures follow Move/Borrow rules
Future, Task	computation or scheduled execution; see Chapter 11

Registering a type name in the compiler does not establish that it is implemented.

Chapter 5 – Operators

<- Chapter 4 · Contents · Next: Chapter 6

Overview

Category | Operators | Native

Arithmetic | + - * / % (binary), - (unary) | yes

Comparison | == != < > <= >= | yes

Logical | && || ! | yes (short-circuiting)

Bitwise | & | ^ < > | yes

Bitwise NOT | ~ | no (NX2002)

Assignment | = += -= *= /= %= | yes

Other assignments | &= |= ^= <<= >>= | frontend

Increment/decrement | ++ -- (pre/post) | no (NX2002); use a += 1 instead

Ternary | condition ? a : b | yes

Other tokens | . , ; : :: -> => ( ) [ ] { } | syntax

Precedence (high -> low), associativity

Level | Operators | Associativity

11 | * / % | left

10 | + - | left

9 | << >> | left

8 | < <= > >= | left

7 | == != | left

6 | & | left

5 | ^ | left

4 | | | left

3 | && | left

2 | || | left

2/1 | ?: (ternary) | right

1 | = and compound assignments | right

Unary operators (- ! ~) bind more tightly than all binary operators. Source: parser (compiler/src/parser/mod.rs).

Behavior

```
public int main() {
    int a = 17;
    return a % 5 + (-a) / 4;     // 2 + (-4) = -2 (division truncates toward 0)
}
```

- Integer division rounds toward zero: -7 / 2 yields -3.
- &&/|| evaluate the right operand only when needed: a != 0 && 10 / a > 1 is safe when a == 0.
- Overflow: Default is trap. add(2147483647, 1) terminates the program with exit code -1073741795 (0xC000001D). With --overflow-checks=false, arithmetic wraps. Division by zero

also traps.

Type compatibility

Both operands of an arithmetic/comparison operator must have the same numeric type (NX1102). Conditions must be bool. Only == != = may be overloaded (static, two parameters, bool return; frontend only).

Common errors

- a++ in native code -> NX2002 Operator 'post++' wird im aktuellen MIR-Subset nicht unterstützt.
- Mixing int and uint/long without a cast.

Operator overloading

Only ==, !=, , = can be overloaded. Each overload is static, has exactly two parameters, and returns bool:

```
public class Vector3
{
    public static bool operator ==(Vector3 left, Vector3 right)
    {
        return left.x == right.x && left.y == right.y && left.z == right.z;
    }
}
```

The frontend checks the operator list, static, parameter count, and bool return type. Planned: checking the relationship of the parameters to the defining type and consistency (==/!=, sensible ordering); the compiler cannot prove mathematical consistency of arbitrary implementations.

Not overloadable: + - * / %, & | ^ ~, ! && ||, >, ++ --, indexing, calls, member access, assignment, and custom operator symbols. && and || retain short-circuit evaluation. ? is only part of the ternary operator c ? a : b; there is no ? error-propagation operator.

Chapter 6 – Control Flow

<- Chapter 5 · Contents · Next: Chapter 7

All constructs in this chapter are verified in examples/language/03_conditions.nx and 04_loops.nx and run natively.

if / else

```
if (bedingung) { ... } else if (bedingung) { ... } else { ... }
```

The condition is in parentheses and must be bool. Braces may be omitted for a single statement (if (a == 1) return 5; tested).

```
public int classify(int score) {
    if (score >= 90) {
        return 1;
    } else if (score >= 50) {
        return 2;
    }
    return 3;
}
```

Nesting is unlimited (nested_if tested). Common error: non-bool condition -> NX1102.

while

```
while (bedingung) { ... }
```

Checks the condition before each iteration. while (true) with break is allowed. There is no do … while.

for

```
for (mut int i = 0; i < 10; i += 1) { ... }
```

Classic form: initialization; condition; step. The loop variable must be mut (otherwise NX1200). i++ as a step fails natively; use i += 1. foreach / for (T v in seq) is frontend-only.

break and continue

break exits the innermost loop; continue jumps to the next iteration. Outside a loop: NX1301 (break) or NX1302 (continue).

return

return expression; ends the function. Functions with a return type require a value (return; -> NX1103 Nicht jeder Kontrollfluss ? liefert einen R?ckgabewert); every control-flow path must return (NX1103 Nicht jeder Kontrollfluss ? liefert einen R?ckgabewert).

Ternary

int t = sum > 5 ? 1 : 2; – right-associative and available natively.

Other constructs (frontend only)

switch/case/default -> build error NX2001 Diese Kontrollflussanweisung wird in der aktuellen Lowering-Stufe nicht unterstützt; try/catch -> likewise; match is partially parsed (arm syntax is not accepted in the test, NX0100).

Chapter 7 – Functions and Methods

<- Chapter 6 · Contents · Next: Chapter 8

Function definition

```
[Modifier] Rückgabetyp name(Typ param1, Typ param2) { Rumpf }

public int add3(int a, int b, int c) {
    return a + b + c;
}
```

- Modifiers: public, private, and static are tested natively (private int helper(), public static int twice(int n)).
- Parameters are immutable (a = a + 1; -> NX1200); mut parameters do not exist (NX0100 Typname erwartet).
- Every function with a non-void return type needs a body and a return on every path (NX1103). A signature without a body -> NX1005.

Calls and recursion

Natively, calls are limited to direct calls to free, non-generic functions. Definition order does not matter (mutual recursion even/odd tested).

```
public int fib(int n) {
    if (n < 2) { return n; }
    return fib(n - 1) + fib(n - 2);
}
```

Wrong number of arguments: NX1107 Keine Überladung von 'f' akzeptiert 2 Argumente. Kandidaten: f(int). Unknown function: NX1004.

Return values

Exactly one value is returned via return. String literals can be assigned to local variables, passed as parameters, and returned by internal functions. void functions cannot be called in native builds, except Console.write/writeLine (NX2002 Void-Aufrufe sind im MIR-Subset nur für Console.write und Console.writeLine verfügbar). String parameters/returns and bool, int, uint are supported natively; the string ABI in extern "C" is not supported.

Local variables

They are in scope from their declaration to the end of the block and are separate for each call (Chapter 4).

Methods and visibility (frontend only)

Methods may appear in class/struct; public/private/protected/internal are parsed. The call c.inc() passes check, but the build rejects classes/new (NX2001; see Chapter 8). Constructors are named after the type; signature and overload checks are missing.

Limitations

No generics, lambdas, native overload resolution, function pointers, or default parameters.

Design rules and frontend status

- The C#-like declaration (public int add(int a, int b)) is the only form; fn name(...) -> T is not part of the language. Constructors are named after the type and have no return type.
- Generic parameters follow the name: public T identity(T value).
- Overloads differ by parameter types, not only by return type. For otherwise valid overloads, unsuffixed integer literals prefer integer parameters over floating-point parameters; equally ranked candidates remain ambiguous (NX1107).
- The frontend detects missing return paths, generic method calls, and type-argument substitution in simple signatures; a non-static method call through the type name is rejected.
- Planned: full generic inference, constraint checking, cross-module resolution, and code generation.

Chapter 8 – Structs and Data Structures

<- Chapter 7 · Contents · Next: Chapter 9

> Status: frontend only. struct, class, new, field access, and methods pass nortexc check, but nortexc build reports NX2001 Dieser Ausdruck wird in der aktuellen Lowering-Stufe nicht unterstützt. They do not produce an EXE. Verified with 07_struct_frontend.nx and 08_class_frontend.nx.

Struct definition

```
public struct Point {
    public int x;
    public int y;
}
```

Fields have a type and visibility; public int value = 0; allows an initial value (class field, tested).

Initialization and access

```
mut Point p = new Point();
p.x = 3;
p.y = 4;
```

Assignment to a field of an immutable binding is rejected: NX1201 Die Veränderung über ein Member oder einen Index erfordert einen exklusiven &mut-Borrow. Therefore use mut Point p.

Classes with methods

```
public class Counter {
    public int value = 0;
    public void inc() { value = value + 1; }
    public int get() { return value; }
}
```

A constructor such as public Counter(int x) { this.v = x; } is parsed; field assignments fail during lowering (NX2001 Zuweisungen sind nur an lokale Namen ? zul?ssig…).

Other declarations

record, interface, trait, and enum (public enum Color { Red, Green }) are parsed. Color c = Color.Red; fails natively. Inheritance : Base, IContract is accepted, but contract checking is missing. Generic types and where clauses are unsupported.

Arrays and collections

Array types T[], [T; N] are parsed in signatures; array creation new T[n] is missing (NX0100). Arrays are not allowed natively (NX2002 Parametertyp 'int[]' liegt außerhalb des skalaren MIR-Subsets). Collections exist only as .nx drafts in standard-library/collections/.

Nesting

Nested structures are planned in the type model (type_system/struct_layout.rs computes layouts), but are not executable.

Classes, inheritance, generics, collections (design target)

The parser and initial declaration checks exist; constructor resolution, receiver borrowing, full inheritance, interface/trait checking, and trait coherence are missing. Everything below is planned:

- Class: nominal type with identity; new creates an instance with exactly one owner. Assignment/passing moves the owner (no copying); sharing uses Rc/Arc. The receiver contract for instance methods is undecided (Chapter 15).
- Struct: value type, Move by default; copying only with a proven Copy property; heap allocation only explicitly via Box or similar.
- Record: immutable value type with structural equality (field types must support equality); does not inherit from classes.
- Interface: signatures only, no fields or default implementations. Trait: capability contract with default methods and associated types. Both appear in the type's : list; there is no impl Trait for Type.
- Inheritance: at most one base class, with any number of interfaces/traits; no multiple class inheritance; structs/records do not inherit from classes.
- Generics: Container, identity; the initial strategy is monomorphization with deduplication of identical instantiations. Bounds use where T: Drawable (multiple bounds with +); each package may have only one implementation per trait/type pair, and either the trait or type must be in the current package (coherence). Current status: the parser knows generic parameters; semantics checks only the argument count of known types.
- Collections (System.Collections, API signatures only in standard-library/collections/*.nx): List (convenient, amortized O(1) append), Vec (low-level, explicit capacity; no implicit conversion to List), Dictionary, HashSet, Queue, Stack. Lengths and indices are usize; read access returns Option, mutable access returns Option. Memory management, hashing, and iterators are missing.

Chapter 9 – Memory Management

<- Chapter 8 · Contents · Next: Chapter 10

Current status

Native NORTEX programs currently work exclusively with scalars on the stack or in registers. There is no heap allocation, deallocation, or pointer that a program can write.

Topic	Status
Stack	Scalar locals and parameters; recursion uses the system stack (stack overflow behavior is not documented/tested).
Heap / new	Frontend accepts new T(); build: NX2001. No allocator is connected.
Deallocation / destructors	not implemented (destructors are unsupported)
Pointers / unsafe	unsafe { } is not parsed; raw pointers are missing
Box, Rc, Arc	design documents only (ownership.md)
Garbage collector	deliberately none

Language goal: Ownership

The target model (ownership.md): one owner per value, Move on passing, &T/&mut T borrows, and explicit clone(). The compiler partially checks these rules (MIR/semantics): moves from local bindings, conflicts between &/&mut, and &mut from an immutable binding (NX1200/NX1201). Full lifetime/region inference is missing.

Safety mechanisms (implemented)

- Immutable by default; mutation only via mut.
- No null in safe code.
- Overflow and division-by-zero trap (exit code -1073741795).
- Scalars are Copy; aliasing problems are impossible.

Lifetime

Local variables live until the end of their block or function call.

Runtime

runtime/ contains Rust prototypes (allocation, memory, exceptions, async); they are not connected to the compiler. Only console output is connected (see Chapter 10).

Planned memory model (design)

No garbage collector; Ownership and Borrowing: Moves transfer the owner, &T allows shared read-only borrowing, &mut T an exclusive mutable borrow, and no reference outlives its owner. Raw pointers are allowed only in unsafe. Span is tied to a borrow lifetime and may not be stored beyond the memory it refers to; Memory owns its memory. Memory is freed through the allocator that provided it. Full Move/Borrow/lifetime checking is not implemented; see ownership.md for details.

Chapter 10 – Input and Output

<- Chapter 9 · Contents · Next: Chapter 11

Console output

```
Console.write("Text without a line break");
Console.writeLine("Text with a line break");
```

Both calls accept exactly one value of type string in native builds. String literals and string variables can be printed. The runtime receives the UTF-8 data pointer and byte length through an ABI call; this is not a general C-ABI string binding.

```
using System;

public int main()
{
    string greeting = "Hello";
    Console.write(greeting);
    Console.writeLine(" NORTEX");
    return 0;
}
```

Internal string values use a 16-byte slice descriptor (data pointer and byte length), passed as an address according to the Windows x64 ABI. String literal bytes and descriptors reside read-only in the COFF data section. Empty strings and embedded NUL bytes are handled correctly based on the length.

```
using System;

public int main() {
    Console.write("a");
    Console.writeLine("b\tc \"quoted\" \\");
    return 0;
}
```

Output: ab?c "quoted" \. Escapes: \n \r \t \0 \" \\. An empty string is allowed. Unicode in a literal is output as UTF-8 bytes; correct rendering depends on the console code page (not systematically tested).

Not supported (NX2001/NX2002 in each case)

Attempt | Message

Console.writeLine(5); | Argument must be of type string

Console.readLine() | only direct free functions and console output

String concatenation

Console.writeLine("Hello " + name); is allowed: + between two string values creates a new heap string through nortex_string_concat (Windows HeapAlloc). It is freed at scope exit through nortex_string_free (drop in codegen; see 04-data-types.md). Top-level strings (const string/mut string) and extern "C" calls with a string argument (char*, NUL-terminated) are also possible; see 04-data-types.md.

Console input, formatting, and interpolation are not implemented. There is no dedicated API for error output (stderr).

Return values as an output channel

Since numbers cannot be printed, example programs return results as an exit code (return countPrimes(100); -> 25). In PowerShell: $LASTEXITCODE.

Standard library

standard-library/system/console.nx and io/mod.nx are drafts with no compiler integration.

Strings (design target)

string is UTF-8 and owns its memory. length() counts bytes (not scalar values or grapheme clusters); indexing is byte-oriented and returns byte; chars() iterates Unicode scalar values (char, 32-bit), while bytes() iterates UTF-8 bytes. Strings must remain valid UTF-8. Planned API: length(), isEmpty(), contains(), startsWith(), endsWith(), replace(), split(), trim(), toUpperCase(), toLowerCase(), chars(), bytes(). A public StringBuilder is not planned. The type checker accepts string + string and string + scalar primitives (result string); formatting is not yet available end-to-end.

Chapter 11 – Errors and Debugging

<- Chapter 10 · Contents · Next: Chapter 12

Format

```
error[NX1102]: Typfehler in Variableninitialisierung: erwartet 'float', erhalten 'double'.
 --> datei.nx:7:19
   |
 7 |     float f = 2.5;
   |               ^^^
```

Messages are in German; codes (NXnnnn) are stable. Severity: error or warning. nortexc exits with code != 0 on errors and does not write an EXE.

Error classes with codes actually observed

Class | Code | Example / fix

Lexical/syntax | NX0100 | int a = ; · @ · missing ; · unterminated string · unknown escape \q. Fix the syntax.

Duplicate declaration | NX1001 | int a twice in one scope -> rename it.

Name resolution | NX1003, NX1004 | Unknown namespace/identifier -> declare it or correct the name.

Member declaration | NX1005 | Function without a body -> add { ... }.

Types | NX1100, NX1102 | int a = "text"; -> use a matching type; literal out of range.

Return | NX1103 | Not all paths return -> add return.

Overload/call | NX1107 | Wrong number of arguments.

Mutability | NX1200, NX1201 | Assignment to immutable binding -> make it mut.

Control flow | NX1301, NX1302 | break/continue outside a loop.

Style warning | NX0004 | Constant not UPPER_SNAKE_CASE, variable not camelCase.

Backend subset | NX2001, NX2002 | Construct not supported by native lowering (see below).

Codegen/linker | (no code) | “exactly one parameterless int-main function”; LNK2019 if main is missing.

> Difference from the code table: diagnostic-codes.md reserves NX0001–NX0003 for lexer errors. The current lexer actually reports these cases as NX0100. Observed behavior is authoritative.

Runtime errors

There are no runtime error messages or exceptions. An overflow or division-by-zero trap terminates the program with exit code -1073741795 (0xC000001D, illegal instruction) and no text.
--overflow-checks=false disables the overflow trap.

Known compiler bug

Programs with long locals and division/comparison against an int literal can trigger a panic in the Cranelift frontend (frontend.rs) instead of a diagnostic. Workaround: use long only as a parameter/return type.

Strategies

1. nortexc check file.nx for frontend-only errors.
2. NX2001/NX2002 mean “not native yet,” not “invalid code”: switch to supported constructs (README).
3. nortexc mir file.nx displays optimized MIR.
4. Return results through the exit code (Chapter 10).

Error handling (design and frontend status)

NORTEX distinguishes optional values (Option: Some/None), expected errors (Result: Ok/Err), and exceptions (throw, try/catch/finally). There is no ? operator. Option and Result are intended to be must_use; an unused result must be handled, propagated, or discarded with discard(value).

```
public Result<int, string> divide(int first, int second)
{
    if (second == 0) { return Err("Division by zero"); }
    return Ok(first / second);
}
```

Current frontend status: using a Result as a standalone expression statement reports NX1108; non-exhaustive enum match arms report NX1109 (_ is the fallback). Option is not checked for must_use. Planned: discard, native exception unwinding, RAII cleanup, complete match, exceptions as status values at the C-ABI boundary; checked/unchecked behavior is undecided (Chapter 15).

Asynchrony (planned)

Goal: async functions become suspendable state machines; await does not block a worker. Future is the computation, Task the execution scheduled by the runtime.

```
public async Task<string> loadText(string path)
{
    return await File.readText(path);
}
```

The parser knows async/await; the compiler does not lower them into the runtime. The Rust prototype runtime/src/async/ has a Waker-based scheduler with a FIFO queue and block_on (for entry points/tests only). Work stealing, timers, a cancellation API, an IOCP reactor, and Send/Sync analysis are missing.

Chapter 12 – Compilation and Technical Architecture

<- Chapter 11 · Contents · Next: Chapter 13

Pipeline

```
.nx -> Lexer -> Parser/AST -> semantische Prüfung -> HIR -> MIR
    -> MIR-Validierung + Optimierung -> Cranelift -> Windows-x64-COFF (.obj)
    -> MSVC-Linker (+ NORTEX-Runtime) -> .exe
```

Stage | Source | Purpose

Lexer | compiler/src/lexer/mod.rs | ASCII identifiers, keywords, numbers, strings/chars, symbols, comment spans (for the IDE)

Parser/AST | parser/mod.rs, ast/mod.rs | Pratt parser for expressions, declarations, statements; errors as NX0100

Semantics | semantic/mod.rs, type_checker.rs | Names, types, mutability, return paths, basic Move/Borrow checks, style warnings

HIR | hir/mod.rs | Resolved, typed intermediate form

MIR | mir/lowering.rs, mir/mod.rs | CFG with blocks; scalar subset; definite-assignment and loan checks; NX2001/NX2002 diagnostics for unsupported constructs

Optimizer | optimizer/mod.rs | Optimizations on validated MIR

Codegen | codegen/mod.rs | Cranelift IR -> COFF object for x86_64-pc-windows-msvc; overflow trap

Linker | linker/mod.rs | Invokes MSVC link.exe, links CRT and runtime; extra arguments from manifest

Driver | driver/mod.rs | Connects the stages

Commands

Command | Effect

nortexc check | Lexer + parser + semantics

nortexc mir | prints validated, optimized MIR

nortexc build [-o …] [--emit-obj] [--overflow-checks[=bool]] | produces .exe or .obj

Backend properties

- Target: Windows x64 (MSVC) only. No Linux/ARM64.
- Entry: CRT's C main; NORTEX int main() returns the process exit code.
- String values are represented as the address of a 16-byte UTF-8 slice descriptor; Console.write(Line) loads its data pointer and byte length for the runtime ABI call.
- Unsupported constructs produce diagnostics rather than being silently ignored (exception: in the tested case, an async declaration builds without effect; unused enum/class declarations are accepted).

Note about older documents

docs/compiler/architecture.md describes an earlier state (“native backend not yet finished”). The native path for the scalar subset is now available and covered by end-to-end tests (sum_to(10) -> exit code 55).

ABI (design target)

extern "C" marks a function with the C ABI; only ABI-compatible parameters/returns are allowed. Structs are C-compatible only with [repr(C)]; string, class owners, generics, exceptions, Task/Future, and arbitrary Result/Option layouts are not automatically C-compatible. Strings are passed as a UTF-8 pointer plus byte length; memory is freed by the allocator that provided it. Symbol names, calling convention, and ABI major version are part of the interface contract. A published binary compatibility guarantee and an FFI test run are missing; authoritative details are in Phase 4.

.ntx package format (planned)

.nx is source code, nortex.toml is the TOML manifest, and .ntx is a custom deterministic binary container (not ZIP and not source code). Reader and writer are not implemented. Design (container format 1, little-endian): 48-byte header with magic NORTEXPK, major/minor version, flags (unknown required flags -> rejection), TOML manifest offset/length, index offset, entry count, and reserved zero bytes; one index entry per file with path length, type, flags, offsets, sizes, and SHA-256, sorted lexicographically; 8-byte-aligned payloads, no compression. Validation: relative UTF-8 paths using / only (no .., no duplicates), every range within the file, size limits before allocation, integer overflow checks, SHA-256 verification (corruption protection, not authentication), optional signatures (method undecided), atomic installation without symlinks. Builds should be byte-for-byte reproducible.

Chapter 13 – Practical Examples

<- Chapter 12 · Contents · Next: Chapter 14

All files are in examples/language/. Each example was built and run with nortexc build (except 07/08, which were checked with check only). Invocation: nortexc build examples\language\NN_name.nx -o out.exe; .\out.exe; $LASTEXITCODE.

1. Hello World – 01_hello.nx

```
using System;

public int main() {
    Console.writeLine("Hello, NORTEX!");
    return 0;
}
```

Prints Hello, NORTEX!, exit code 0. main is the entry point; the return value is the exit code.

2. Variables and calculations – 02_variables.nx

```
public int main() {
    int width = 6;
    int height = 7;
    int area = width * height;
    mut int total = area;
    total += 8;
    total -= 3;
    const int OFFSET = 3;
    return total + OFFSET;
}
```

Values are immutable by default; mut makes total mutable. Calculation: 42 -> 50 -> 47 -> 50. Exit code 50.

3. Conditions – 03_conditions.nx

```
public int classify(int score) {
    if (score >= 90) {
        return 1;
    } else if (score >= 50) {
        return 2;
    }
    return 3;
}

public int main() {
    Console.writeLine("Classify the value 72");
    return classify(72);
}
```

72 satisfies only the second condition. Output: Classify the value 72, exit code 2.

4. Loops – 04_loops.nx

```
public int main() {
    mut int sum = 0;
    for (mut int i = 1; i <= 10; i += 1) {
        if (i % 2 != 0) {
            continue;
        }
        sum += i;
    }

    mut int n = 0;
    while (true) {
        n += 1;
        if (n == 5) {
            break;
        }
    }
    return sum + n;
}
```

for sums the even numbers (2+4+6+8+10 = 30); continue skips odd numbers; while (true) ends at 5 via break. Exit code 35.

5. Functions and recursion – 05_functions.nx

```
public int gcd(int a, int b) {
    if (b == 0) {
        return a;
    }
    return gcd(b, a % b);
}

public int fib(int n) {
    if (n < 2) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

public int main() {
    return gcd(48, 18) + fib(10);
}
```

gcd(48,18) = 6, fib(10) = 55. Exit code 61.

6. Complete application: prime numbers – 06_primes.nx

```
using System;

public bool isPrime(int n) {
    if (n < 2) {
        return false;
    }
    mut int d = 2;
    while (d * d <= n) {
        if (n % d == 0) {
            return false;
```

```
        }
        d += 1;
    }
    return true;
}

public int countPrimes(int limit) {
    mut int count = 0;
    for (mut int n = 2; n <= limit; n += 1) {
        if (isPrime(n)) {
            count += 1;
        }
    }
    return count;
}

public int main() {
    Console.writeLine("Count primes up to 100");
    return countPrimes(100);
}
```

Combines bool functions, loops, early returns, call chains, and console output. There are 25 primes <= 100. Exit code 25.

7. Structs – 07_struct_frontend.nx (check only)

```
public struct Point {
    public int x;
    public int y;
}

public int main() {
    mut Point p = new Point();
    p.x = 3;
    p.y = 4;
    return p.x + p.y;
}
```

nortexc check succeeds; nortexc build -> NX2001. Shows the intended syntax, but is not yet executable.

8. Methods – 08_class_frontend.nx (check only)

Class Counter with inc()/get(); same status as Example 7 (see Chapter 8).

9. Heap allocation

Not executable: new always produces NX2001 in native builds. Therefore, there is no runnable heap example (Chapter 9).

Chapter 14 – Language Reference

<- Chapter 13 · Contents · Next: Chapter 15

Keywords

Source: is_keyword in compiler/src/lexer/mod.rs.

Group | Words | Usable natively

Declarations | using namespace class struct record interface trait enum | declarations accepted only

Modifiers | public private protected internal static abstract sealed partial extern unsafe const virtual override async readonly | public private static const

Mutability | var mut ref out params | mut

Control flow | return throw if else while for in switch case default break continue try catch finally match | return if else while for break continue

Literals | true false null | true false (null forbidden)

Expressions | new await this base operator is as typeof sizeof where | no

Reserved | let | no (NX0100)

Types | sbyte short int long i128 byte ushort uint ulong u128 nint nuint isize usize float double decimal bool char string void | int bool uint short ulong long (limitations: Chapter 4)

Not keywords: Console, System, get, set, do, foreach, fn.

Tokens

Identifier, Keyword, Number (decimal, 0x, 0b, exponent, suffix), StringLiteral, CharacterLiteral, Symbol, EndOfFile. Comments are captured separately.

Operators and symbols

+ - * / % · == != = · && || ! · & | ^ ~ > · = += -= *= /= %= &= |= ^= >= · ++ -- · ? : · . , ; ( ) [ ] { } · ->
=> ::. Precedence: Chapter 5.

Grammar constructs (native)

```
Datei        = { Using } { Funktion } ;
Using        = "using" Name ";" ;
Funktion     = { Modifier } Typ Name "(" [ Typ Name { "," Typ Name } ] ")" Block ;
Block        = "{" { Anweisung } "}" ;
Anweisung    = Deklaration | Zuweisung ";" | Aufruf ";" | If | While | For
             | "break" ";" | "continue" ";" | "return" [ Ausdruck ] ";" | Block ;
Deklaration  = [ "mut" | "const" ] Typ Name "=" Ausdruck ";" ;
If           = "if" "(" Ausdruck ")" Anweisung [ "else" Anweisung ] ;
While        = "while" "(" Ausdruck ")" Anweisung ;
For          = "for" "(" Deklaration Ausdruck ";" Zuweisung ")" Anweisung ;
Ternär       = Ausdruck "?" Ausdruck ":" Ausdruck ;
```

This is a simplified view of the natively supported subset. The complete frontend grammar: grammar.ebnf.

Supported statements

Local declaration, assignment (simple/compound), function call, Console.write(Line), if/else, while, for, break, continue, return, block.

Built-in functions

Function	Signature	Limitation
Console.write	(string)	exactly one string value
Console.writeLine	(string)	exactly one string value

There are no other built-in functions (Console.readLine cannot be resolved).

Diagnostic codes (observed)

NX0004 NX0100 NX1001 NX1003 NX1004 NX1005 NX1100 NX1102 NX1103 NX1107 NX1200 NX1201 NX1301 NX1302 NX2001 NX2002. See Chapter 11.

Relevant limitations

intrinsic is a context-sensitive modifier only for files under standard-library/; in application code it is rejected with NX1005. No intrinsic/runtime symbol binder exists.

See the feature overview. Also: Windows x64 only; main must be public int main() without parameters; module system has no qualified names or visibility (using uses a flat namespace); long locals with division can crash codegen.

Chapter 15 – FAQ and Best Practices

<- Chapter 14 · Contents

Common beginner mistakes

Error	Cause / fix
NX1200 for i += 1 in for	Declare the loop variable mut: for (mut int i = 0; …)
NX1200 when changing a parameter	Parameters are immutable; create a local mut copy.
NX2001/NX2002 during build	Construct is frontend-only (classes, heap strings, var, switch, …).
“exactly one parameterless int-main function”	Use public int main().
a++ fails	Use a += 1;
Cannot print a number	Return the result as an exit code.
let x = 1;	Specify the type: int x = 1;
NX1102 for uint + int	Use matching types; there are no casts.

FAQ

Why doesn't my class run? class/struct/new are supported by the frontend, but not by native lowering. Can I store strings? Yes. Literals, local variables, top-level const/mut string, internal parameters, and returns use UTF-8 slices; a + b creates heap strings (without deallocation). How do I debug? Use nortexc check, nortexc mir, and exit codes; there is no debugger. Is Linux supported? No, Windows x64 only. Are there modules? Yes: using util; resolves util.nx or util/mod.nx (flat namespace); see the implemented specification.

Recommended practices

- Names: functions/variables camelCase, constants UPPER_SNAKE_CASE, types PascalCase (matches NX0004 warnings).
- Mutability: use mut only where needed.
- Structure: small functions with a single return path; keep main as an orchestrator.
- Error checking: check for 0 before division (otherwise it traps rather than reporting an error).
- Types: prefer int/bool in native code; use explicit types instead of var.
- Verification: build examples after each change and check the exit code.

Performance

The backend optimizer operates on MIR; recursion (fib) is not optimized (no tail-call proof). Overflow checking has some cost; use --overflow-checks=false only deliberately, as wraparound then occurs without a warning.

Larger projects

Build with nortex.toml (Chapter 2). Multiple .nx files can be combined with using (flat namespace, no visibility checks between modules; see the implemented specification).

Open decisions (project owner)

These items have not been approved; the compiler must not treat them as final:

1. License: The repository header says MIT; this is not confirmed legal approval.
2. Exceptions: checked or unchecked behavior and the native unwinding mechanism are undecided. Exceptions must never escape uncontrollably across a C-ABI boundary.
3. Arithmetic: Whether checked arithmetic is the release default is undecided (currently configurable through overflow checks; see the manifest).
4. Method receiver: &self, &mut self, or another instance-method contract is undecided.
5. Backend: The backend is Cranelift with MSVC link.exe; LLVM is not binding and at most a non-normative prototype.

Earlier design rules superseded by the current implementation (bindings “mutable by default,” .nortex manifest, lld-link linker, “no native EXE”) no longer apply.